LangGraph Learning Notes

A practical, lesson-by-lesson guide to building a personal AI productivity agent

Course roadmap

Lesson	Topic	Status
1	Understanding LangGraph architecture	Covered
2	State, Nodes and Edges	Covered
3	Conditional edges and routing	Covered
4	Loops and agent execution	Covered
5	LLM integration	Upcoming
6	Tool calling	Upcoming
7	Memory and persistence	Upcoming
8	Human-in-the-loop	Upcoming
9	Multi-agent systems	Upcoming
10	Building your personal AI agent	Upcoming

This is a cumulative set of notes. It currently contains the material covered in Lessons 1–4. Lessons 5–10 are listed as the remaining course plan and will be added as they are taught.

How to use these notes

The examples build toward a single project: a private, Telegram-accessible productivity agent that can manage tasks, reminders, daily plans and goals. The early lessons use small deterministic examples so the graph mechanics are clear before adding an LLM.

Target application

• Telegram is the user interface for sending commands and receiving alerts.

• FastAPI exposes the agent through an HTTP API.

• LangGraph coordinates state, routing, model calls and tools.

• PostgreSQL stores application data and can also back persistent graph checkpoints.

• A local model served by Ollama can handle natural-language requests on the laptop.

• A scheduler such as APScheduler triggers reminders; the LLM should not be responsible for exact-time delivery.

Important distinction: LangGraph structures the agent's execution. It does not automatically provide a database, a scheduler, an API server, or persistent storage.

Lesson 1 — Understanding LangGraph architecture

What is LangGraph?

LangGraph is a framework for building stateful, multi-step applications and agents as graphs. Instead of writing one large function that handles every situation, you split the work into nodes and define how execution moves between them.

The three fundamental concepts

• **State:** the shared data carried through the graph. It can contain the user's message, detected intent, extracted task details, tool results and final response.

• **Node:** a function that performs one unit of work. A node receives the current state and returns updates to it.

• **Edge:** a connection that determines which node executes next. An edge can be fixed or conditional.

Example workflow

```
User message
    |
    v
Understand request
    |
    v
Classify intent
    |
    +---- create task ----> Create task ----> Save task
    |
    +---- ask question ---> Generate answer
    |
    v
Return response
```

A minimal graph

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class AgentState(TypedDict):
    user_input: str
    intent: str
    response: str

def classify(state: AgentState):
```

```
    text = state["user_input"].lower()
    if "remind" in text or "task" in text:
        return {"intent": "task"}
    return {"intent": "general"}

def respond(state: AgentState):
    return {"response": f"Intent detected: {state['intent']}"}

builder = StateGraph(AgentState)
builder.add_node("classify", classify)
builder.add_node("respond", respond)
builder.add_edge(START, "classify")
builder.add_edge("classify", "respond")
builder.add_edge("respond", END)

agent = builder.compile()
result = agent.invoke({"user_input": "Remind me to study"})
print(result["response"])
```

The graph is compiled before it is invoked. **START** marks the entry point and **END** marks completion.

Key takeaway

Think of LangGraph as the control-flow layer of your agent: state carries information, nodes do work, and edges control execution.

Practice task

Extend the example with three intents: create_task, delete_task and general. Add one node for each intent. For this first exercise, use simple keyword checks rather than an LLM.

Lesson 2 — State, Nodes and Edges

State is shared data

State is the structured data passed from node to node. A node reads the fields it needs and returns a dictionary containing the fields it wants to update.

TypedDict

```
from typing import TypedDict

class AgentState(TypedDict):
    user_input: str
    intent: str
    response: str
```

TypedDict describes the expected structure to Python's type checker. It does not perform runtime validation by itself.

Nodes return partial updates

```
def classify(state: AgentState):
    text = state["user_input"].lower()
    if "remind" in text:
        return {"intent": "create_reminder"}
    return {"intent": "general"}
```

The node does not need to return every field. LangGraph merges the returned update into the existing state. By default, a field update replaces that field's previous value.

Reducers

A reducer defines how an incoming update is combined with the existing value for a state field. For example, `operator.add` concatenates lists.

```
import operator
from typing import Annotated, TypedDict

class AgentState(TypedDict):
```

```
    tasks: Annotated[list[str], operator.add]
```

If the existing tasks are ["Read"] and a node returns ["Revise"], the reducer combines them into ["Read", "Revise"].

When to use a reducer

• Use a reducer when updates should be accumulated, such as appending messages or collecting results.

• Use normal replacement when a node returns the complete new value, such as a task list after deletion.

• Concurrent nodes updating the same state key need a suitable reducer to define how their updates combine.

The add/delete distinction

```
# Add by returning the complete updated list (replacement semantics)
def add_task(state):
    tasks = state["tasks"] + ["Study graphs"]
    return {"tasks": tasks}

# Delete by returning the complete updated list
def delete_task(state):
    tasks = [t for t in state["tasks"] if t != "Study graphs"]
    return {"tasks": tasks}
```

Do not use an append-only reducer such as operator.add for a field that must also support replacing the whole list for deletions. Choose state semantics that fit all operations.

Messages and add_messages

```
from typing import Annotated
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
```

`add_messages` is a message-aware reducer commonly used for conversation history. It handles message updates and IDs more appropriately than blindly concatenating lists.

Practice task

Build add_task, delete_task and view_tasks nodes. Keep tasks in a list, use replacement semantics for list changes, and route to the appropriate node.

Lesson 3 — Conditional edges and routing

Fixed edges vs conditional edges

• **Fixed edge:** always sends execution to the same next node.

• **Conditional edge:** calls a routing function to choose the next node based on the current state.

```
builder.add_edge("classify", "respond")

def route_request(state):
    if state["intent"] == "create_task":
        return "task"
    if state["intent"] == "delete_task":
        return "delete"
    return "general"

builder.add_conditional_edges(
    "classify",
    route_request,
    {
        "task": "create_task",
        "delete": "delete_task",
        "general": "general_response",
    },
)
```

How routing works

The routing function reads state and returns a label. The mapping passed to add_conditional_edges maps that label to a graph node name. The labels do not have to be identical to node names.

Example task-management graph

```
START
  |
classify
  |
  +---- create_task ----> create_task ----+
  |                                       |
  +---- delete_task ----> delete_task ----+--> END
  |                                       |
  +---- view_tasks -----> view_tasks -----+
  |
  +---- general --------> general_response -> END
```

Command: update state and route together

LangGraph's Command can be used by a node to update state and choose the next node in the same operation.

```
from langgraph.types import Command

def classify(state):
    intent = detect_intent(state["user_input"])
    return Command(
        update={"intent": intent},
        goto="create_task" if intent == "create_task" else "general_response",
    )
```

Use one routing mechanism for a particular decision. Avoid combining a Command-based goto with a separate conditional edge from the same node for the same routing decision, because that can create unintended execution paths.

Handling invalid requests

A robust router should include a fallback path for unknown or unsupported intents. The fallback can ask the user to clarify rather than silently choosing an unrelated operation.

Practice task

Add view_tasks and update_task intents to your graph. Route all supported intents and provide a fallback response for invalid requests.

Lesson 4 — Loops and agent execution

What is a cyclic graph?

A graph is cyclic when execution can return to a node that has already run. This is useful when an agent needs to take an action, inspect the result and decide whether another action is necessary.

Basic retry loop

```
def generate_plan(state):
    attempt = state["attempt"] + 1
    plan = make_plan(attempt)
    return {"plan": plan, "attempt": attempt}

def validate_plan(state):
    valid = len(state["plan"]) >= 3
    return {"valid": valid}

def route_after_validation(state):
    if state["valid"]:
        return "finish"
    if state["attempt"] >= 3:
        return "finish"
```

```
    return "retry"

builder.add_conditional_edges(
    "validate",
    route_after_validation,
    {"retry": "generate", "finish": "final_response"},
)
```

The example assumes the graph has nodes named generate, validate and final_response, and that make_plan is implemented. It illustrates the control flow rather than a complete runnable application.

The agent execution loop

```
LLM decides what to do
       |
       v
Does it need a tool?
   /          \
 no            yes
 |              |
 v              v
Respond       Execute tool
                 |
                 v
              Observe result
                 |
                 +----> LLM decides again
```

Workflow vs agent

• **Workflow:** follows a mostly predefined sequence of steps.

• **Agent:** uses model decisions to select actions dynamically, often in a loop.

• LangGraph can implement both patterns.

Recursion limit and stop conditions

```
result = agent.invoke(
    input_data,
    config={"recursion_limit": 10},
)
```

The recursion limit protects execution from running indefinitely. It is not a replacement for explicit stop conditions. Design loops with a success condition, a maximum number of attempts, and error handling.

Practice task

Create a graph with generate_plan, validate_plan and final_response. Require at least three tasks in a valid plan. If validation fails, regenerate the plan, but stop after three attempts.

Lessons 5–10 — Remaining course plan

These lessons have not yet been taught in this conversation. Their sections are reserved for the explanations, code examples and exercises as the course progresses.

Lesson 5 — LLM integration

Connect a chat model to graph nodes. Learn system, human and AI messages, model invocation, prompts and structured outputs.

To be completed as this lesson is taught.

Lesson 6 — Tool calling

Let the model request tools, execute Python functions, return tool results to the model and handle tool errors safely.

To be completed as this lesson is taught.

Lesson 7 — Memory and persistence

Use checkpointers for resumable graph state, thread IDs for conversations, and long-term storage for durable user facts and preferences.

To be completed as this lesson is taught.

Lesson 8 — Human-in-the-loop

Pause execution for user approval or clarification, then resume the graph. Apply approval gates to sensitive actions.

To be completed as this lesson is taught.

Lesson 9 — Multi-agent systems

Separate responsibilities across specialist agents and coordinate them through a supervisor or another orchestration pattern.

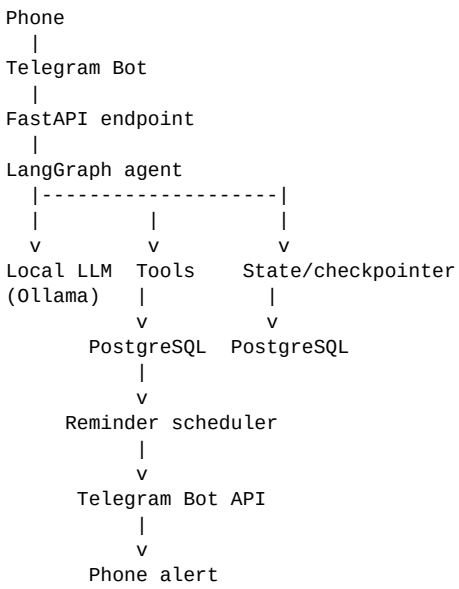
To be completed as this lesson is taught.

Lesson 10 — Building your personal AI agent

Combine the lessons into a Telegram-accessible productivity agent with tasks, reminders, planning, goals, memory, persistence and notifications.

To be completed as this lesson is taught.

Appendix — Target personal-agent architecture



Responsibilities

- **Telegram bot:** receives user messages and sends replies or alerts.
- **FastAPI:** provides the HTTP interface between Telegram and the agent.
- **LangGraph:** manages state transitions, routing, model calls and tool execution.
- **Ollama:** serves a local LLM on the laptop.
- **PostgreSQL:** stores tasks, reminders, goals and persistent graph checkpoints.
- **Scheduler:** triggers reminders at their scheduled times; it should work independently of an LLM response.

For a local-first deployment, the laptop must stay powered on and connected to the internet. A secure tunnel can expose the API without opening router ports. Restrict the bot to the owner's Telegram user ID and protect the API with authentication.